

Table des matières

Contexte	2
Création de la base de données.....	3
Qu'est ce qu'un déclencheur	3
Comment créer un déclencheur	4
Application 1 : Gestion de la casse du texte d'un champ	5
Application 2 : Génération de login/mot de passe.....	5
Application 3 : Archivage automatique.....	5
Application 4 : Mise à jour d'un champ (niveau 1).....	6
Application 5 : Mise à jour d'un champ (niveau 2).....	6
ANNEXE : Protéger les données sensibles d'une base par le hachage	7
Concepts clés.....	7
Mise en œuvre avec SQL Server	8

Cours SQL Server

Déclencheurs (triggers)

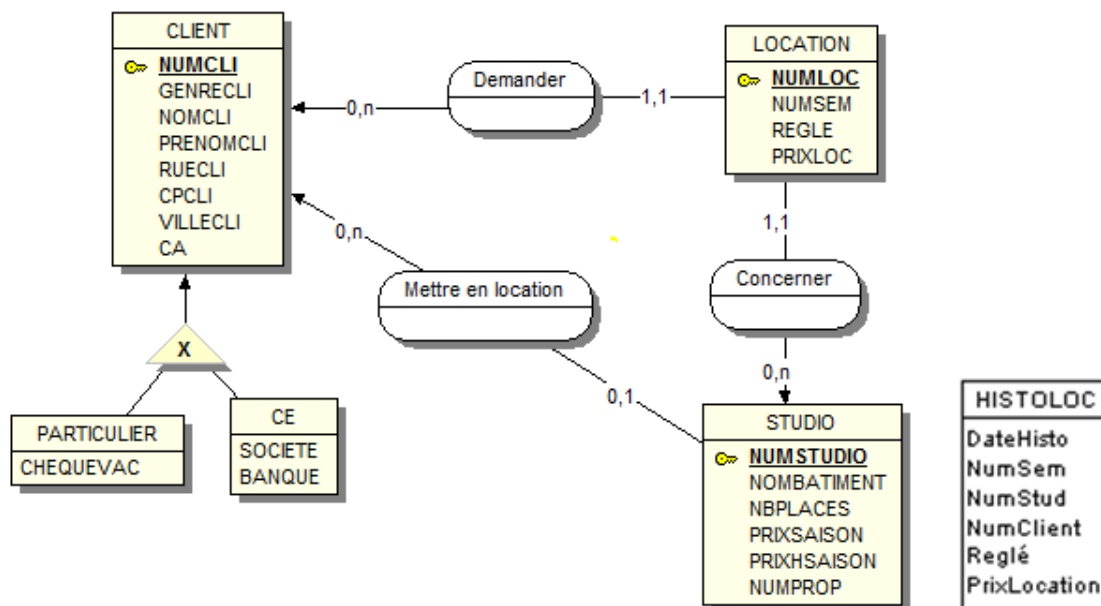
Contexte

L'agence immobilière **Mer et Soleil** propose la location d'appartements sur le Languedoc Roussillon. Elle dispose de différentes agences situées sur Montpellier, La Grande Motte et Agde. Elle souhaite mettre en place une application base de données répartie permettant aux différentes agences de gérer les locations d'appartements.

Suite à l'analyse effectuée, les principales **règles de gestion** suivantes ont été formalisées :

- RG1** On distingue deux types de client : les particuliers et les comités d'entreprise (CE). La notion de client inclut la notion de propriétaire.
- RG2** Le chiffre d'affaires (CA) global réalisé est enregistré dans la fiche du client.
- RG3** Un studio doit être mis en location par un client propriétaire avant d'être loué.
- RG4** Les locations se font à la semaine et comence le samedi.
- RG5** La saison haute va du 01/05 au 15/09.
- RG6** Toute semaine commencée en saison haute est considérée comme haute.
- RG7** Les particuliers peuvent régler à l'aide de chèque vacances.
- RG8** Pour les comités d'entreprise, on conserve le nom de la société et de la banque.
- RG9** En fin d'année, les locations supprimées sont historisées.

Une étude a permis d'établir le **modèle conceptuel de données** suivant :



Le **modèle relationnel** correspondant est :

CLIENT	(<u>NumCli</u> , GenreCli, NomCli, PrenomCli, RueCli, CPcli, VilleCli, CA)
PARTICULIER	(<u>NumCli</u> , ChequeVac)
CE	(<u>NumCli</u> , NomSociete, Banque)
STUDIO	(<u>NumStu</u> , NomBat, NbPlaces, PrixSaison, PrixHSaison, #NumProp)
LOCATION	(<u>NumLoc</u> , #NumStu, #NumCli, NumSem, Reglé, PrixLoc)
HISTOLOC	(<u>id</u> , DateHisto, NumSem, NumStud, NumClt, Reglé, PrixLoc)

Création de la base de données

La création de la base de données correspondante va être créée à partir d'un script :

- Démarrer **SQL Management Studio**
- Créer une nouvelle base de données : **MerEtSoleil_VotreNom**
- Créer une **nouvelle requête**
- Importer, vérifier et exécuter le fichier **Script création Mer Et Soleil**
- Créer une **nouvelle requête**
- Importer, vérifier et exécuter le fichier **Script données Mer Et Soleil**
- Vérifier l'existence des tables et des données

Qu'est ce qu'un déclencheur

Les **déclencheurs** (ou triggers) permettent de mettre en œuvre les **mécanismes d'intégrité complexes (contraintes)** sur une base de données, correspondant à des règles de gestion d'entreprise.

La **prise en compte des contraintes** peut être réalisée à différents niveaux, par :

1. Les **types de données** associés aux champs des tables
Exemple : le chiffre d'affaire d'un client (champ **CA**) est de type **Numérique/réel**
2. Les contraintes sur les champs (**clé primaire, NULL interdit, Index**)
Exemple 1 : chaque client possède un n° client unique
➔ contrainte de **clé primaire** sur le champ **NumCli**

Exemple 2 : pour enregistrer une fiche client, le nom du client est requis
➔ contrainte **NULL Interdit** sur le champ **NomCli**
3. Les **règles de validation de tables** ()
Exemple : le prix hors saison ne peut pas être supérieur au prix en saison
➔ Contraintes **CHECK** entre les champs **PrixHSaison** et **PrixSaison**
4. Les **contraintes d'intégrité référentielle** à la création d'une relation entre tables
Exemple : la location doit être associée à un client (champ **#NumCli**) existant dans la table Client (champ **NumCli**). En conséquence, la suppression d'un client ne sera pas autorisée si des locations lui sont associées.
5. Les **déclencheurs**

Un **déclencheur** est une **procédure stockée** particulière qui contient des instructions **Transact-SQL** simples ou complexes, associée à une **table** ou à une **vue**, qui s'exécute **automatiquement** lorsqu'un événement associé à cette table se produit.

Les événements sont :

- L'insertion (**INSERT**) d'enregistrements.
- La mise à jour (**UPDATE**) de données de champs.
- La suppression (**DELETE**) d'enregistrements.

Un même déclencheur peut être créé pour plusieurs événements simultanément.

Cours SQL Server

Déclencheurs (triggers)

Les types de déclencheurs :

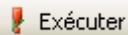
- Les déclencheurs **AFTER** ou **FOR** (synonyme) s'exécute par défaut lorsque l'instruction qui les a lancés est terminée. Ils s'appliquent aux tables, mais pas aux vues.
- **INSTEAD OF** : exécuté à la place d'une opération d'insertion, de mise à jour ou de suppression. Ils sont utiles pour effectuer des contrôles d'intégrité sur les valeurs de données fournies dans les instructions **INSERT** et **UPDATE** et pour spécifier des actions qui permettent aux vues d'être mises à jour, ce qui n'est habituellement pas le cas.

La définition d'un déclencheur crée 2 tables temporaires :

- **inserted** : copie des lignes ajoutées par **INSERT** ou **UPDATE**.
Cette table donne l'image du t-uple avant l'insertion ou la mise à jour.
- **deleted** : copie des lignes supprimées par **DELETE**.
Cette table donne l'image du t-uple avant la suppression.

Comment créer un déclencheur

Pour **créer un déclencheur** :

- Développer l'arborescence de la base de données
- Développer la table (ou la vue) concernée
- Cliquer droit sur Déclencheur et choisir Nouveau déclencheur
- Saisir le déclencheur, le tester , l'exécuter 

Remarques :

- Un déclencheur exécuté sans erreur est associé à l'objet (table ou vue) et apparaît dans l'arborescence
- Pour le modifier, il faut remplacer le mot clé **CREATE** par **ALTER**.
- Par sécurité, enregistrer les déclencheurs dans un fichier texte.

Syntaxe générale :

```
[CREATE] [ALTER] TRIGGER Nom_Trigger
ON [NomTable | NomVue]
[AFTER] [INSTEAD OF] [INSERT, DELETE, UPDATE]
AS
    BEGIN
        Instructions
    END
GO
```

Pour **tester le déclencheur** :

- Ouvrir la table (ou la vue) concernée
- Faire l'opération (insertion, mise à jour ou suppression)
- Vérifier si le déclencheur s'est bien exécuté

Application 1 : Gestion de la casse du texte d'un champ

Ce déclencheur (INSERT, UPDATE) transforme en majuscule les noms de client saisis.

```
CREATE TRIGGER NomEnMaj
ON Client
FOR INSERT, UPDATE
AS
    UPDATE Client
    SET NomCli = UPPER(inserted.NomCli)
    FROM Client, inserted
    WHERE Client.NumCli = inserted.NumCli
```

→ Tester le déclencheur

→ Amélioration : modifier ce déclencheur pour permettre d'avoir dans la table :

- Le nom du client en majuscule
- La première lettre du prénom en majuscule et le reste du prénom en minuscule.

Application 2 : Génération de login/mot de passe

Compléter le trigger précédent pour générer un login et un mot de passe lors de l'insertion d'un nouveau client (voir aide en annexe).

- Ajouter préalablement les 2 champs **login** et **motDePasse** de type VARCHAR(50) dans la table Client.
- Le login (en minuscule) sera constitué de la première lettre du prénom suivie du nom. Exemple : pour Alain DUPONT, le login sera : a.dupont
- Le mot de passe sera haché à l'aide de la fonction HASHBYTES (utiliser les informations de l'annexe pour hacher le mot de passe et le tester)

Application 3 : Archivage automatique

Ce déclencheur (DELETE) copie une ligne supprimée de la table Location, dans la table HistoLoc

```
CREATE TRIGGER HistoLocation
ON Location
FOR DELETE
AS
    INSERT INTO HistoLoc (DateHisto, NumSem, NumStu, NumCli, Reglé, PrixLoc)
    SELECT GetDate(), NumSem, NumStu, NumCli, Regle, PrixLoc
    FROM deleted
```

NB : la fonction **GetDate()** renvoie la date du jour.

→ Tester le déclencheur

Application 4 : Mise à jour d'un champ (niveau 1)

Ce déclencheur (INSERT) met à jour le champ CA de la table Client, lors de l'enregistrement d'une location, à partir du prix de la location récupéré dans la table Studio (PrixSaison).

```
CREATE TRIGGER MajCAClient
ON          Location
FOR          INSERT AS

DECLARE      @PrixStudio INT

-- Récupération du prix saison du studio loué
SELECT      @PrixStudio = PrixSaison
FROM        Studio, inserted
WHERE       Studio.NumStudio = inserted.NumStu

-- Cumul du CA (chiffre d'affaires) pour le client concerné
UPDATE      Client
SET         CA=CA+@PrixStudio
FROM        Client, inserted
WHERE       Client.NumCli = inserted.NumCli
```

➔ Désactiver le déclencheur précédent : Clic droit sur le déclencheur / Désactiver

NB : en SQL, on écrit `DISABLE / ENABLE TRIGGER Nom_trigger ON dbo.NomTable;`

➔ Tester le déclencheur en saisissant une nouvelle location dans la table **Location**

Application 5 : Mise à jour d'un champ (niveau 2)

On vous demande d'améliorer le déclencheur précédent en récupérant le prix de location d'un studio en fonction de la semaine choisie :

- Si la période se situe entre les semaines 19 et 38, le prix saison sera récupéré.
- Si la période se situe avant la semaine 19 ou après la semaine 38, le prix hors saison sera récupéré

Principe algorithmique :

1. Récupérer dans une variable la **période** de location
2. Récupérer le **prix** de location **saison** ou **hors saison** selon la période
3. Mettre à jour le **CA** (chiffre d'affaires=total des locations) de table client en fonction du prix

Utiliser la documentation Transact-SQL fournie en ressources, ou toute ressource technique officielle sur Internet

➔ Tester le déclencheur en saisissant 2 nouvelles locations dans la table **Location**, une en semaine 25 et une en semaine 10.

ANNEXE : Protéger les données sensibles d'une base par le hachage

Concepts clés

Cryptage : Transformation d'un message en clair en un message codé incompréhensible pour qui ne connaît pas le code, de façon à en protéger la confidentialité ou l'authenticité.

La particularité du cryptage, c'est que le message crypté peut être décrypté (**réversibilité**). Pour lire le message, il faut avoir la clé permettant de lui redonner sa forme originale.

Hachage : opération consistant à transformer un message de taille variable en un code de taille fixe en appliquant une fonction mathématique dans le but d'authentifier ou de stocker ce message.

Une chaîne ayant subi un hachage ne pourra pas être décodée (**non réversibilité**).

Salage : Saler un mot de passe consiste à lui concaténer une chaîne de caractères avant de lui appliquer un algorithme de hachage. Pour que la technique soit efficace, il faut que chaque usager ait sa propre clé de salage, que cette clé soit générée aléatoirement et qu'elle soit aussi longue que possible.

La clé de salage sera généralement stockée dans la base de données.

MySQL offre la fonction **UUID()** qui peut être utilisée pour générer une clé de salage. La clé générée avec UUID() n'est pas complètement imprévisible. Elle est cependant suffisamment aléatoire pour offrir le niveau de sécurité espéré.

En PHP, l'utilisation de **password_hash()** est très intéressante puisqu'elle génère une clé de salage qui fait partie intégrante du résultat du hachage.

La fonction **password_verify()** permet de vérifier si le mot de passe en clair correspond à celui de la base de donnée.

Mise en œuvre avec SQL Server

Application exemple : Génération d'un login et d'un mot de passe pour les clients.

Etape 1 : Ajouter 2 champs dans la table Client

- Login : VARCHAR(50)
- Passwd : VARCHAR(128)

Etape 2 : Compléter le déclencheur sur la table Client

```
CREATE TRIGGER [dbo].[AjoutClient]
ON [dbo].[Client]
FOR INSERT
AS
    UPDATE Client
    SET
        NomCli = UPPER(inserted.NomCli),
        login = LOWER(LEFT(inserted.PrenomCli, 1)) + '.' + LOWER(inserted.NomCli),
        passwd = CONVERT(VARCHAR(128), HASHBYTES('SHA2_512', inserted.passwd), 2)
    FROM Client, inserted
    WHERE Client.NumCli = inserted.NumCli;
```

Le **login** est constitué de la première lettre du prenom, d'un point et du nom

Le **password** est haché avec l'algorithme SHA2_512 qui renvoie une donnée cryptée de type binaire. Il convient donc de le convertir en VARCHAR(128).

Etape 3 : Tester le trigger par l'insertion d'un nouveau client

```
-- Ajout d'un client pour test
INSERT INTO Client(Genrecli, NomCli, PrenomCli, RueCli, CpCli, VilleCli, login, passwd)
VALUES ('Monsieur', 'BARD', 'Luc', 'Rue de la paix', '30000', 'Nîmes', 'test1', 'test1')
```

Lors de l'exécution de cette insertion, grâce au trigger, le mot de passe « en clair » qui est défini à « test1 » est haché par la fonction HASHBYTES.

Cours SQL Server

Déclencheurs (triggers)

Etape 5 : Créer et exécuter une fonction SQL Server de test de conformité du login et du mot de passe haché.

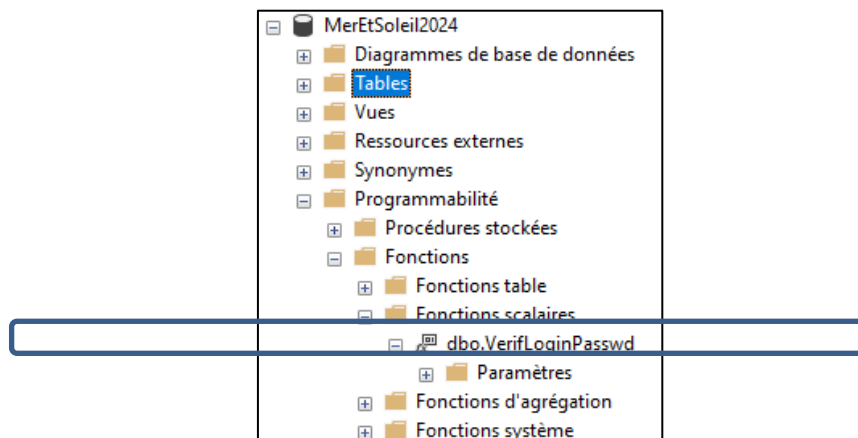
La **fonction** est créée dans **Programmabilité**

Cette fonction reçoit un **login** et **mot de passe en clair** (exemple : **test1**), si le mot de passe haché est conforme, la fonction retourne **1** sinon elle retourne **0**.

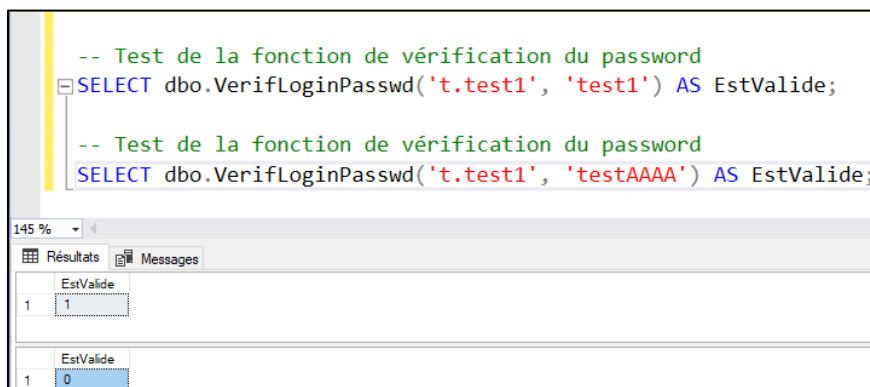
```
CREATE FUNCTION [dbo].[VerifLoginPasswd](@login VARCHAR(50), @password VARCHAR(128)) RETURNS BIT
AS
BEGIN
    DECLARE @estValide BIT = 0;
    -- Hachage du mot de passe fourni
    DECLARE @hashedPassword VARCHAR(128) = CONVERT(VARCHAR(128), HASHBYTES('SHA2_512', @password), 2)

    -- Vérification dans la table Client
    IF EXISTS ( SELECT 1
                FROM Client
                WHERE login = @login AND passwd = @hashedPassword)
    BEGIN
        SET @estValide = 1;
    END
    RETURN @estValide;
END;
```

Vérifier l'existence de la fonction dans **Programmabilité** :



Etape 6 : Utiliser cette fonction pour tester votre login et mot de passe



Référence complémentaire : une autre solution est de gérer le hachage du mot de passe dans le **programme php** avec les fonctions **password_hash** et **password_verify**.

Mais les 2 systèmes (par PHP et par SQL Server) ne sont pas compatibles entre eux.